CSC 436 Assignment

Populating & Querying Your Database

Overview

In this assignment, you will navigate the full lifecycle of database management, starting with the setup and configuration of a new database in cPanel. With a focus on data integrity, you will learn to create tables with primary key, not null, unique, check, and foreign key constraints. Once your database is populated, you will dive into querying, mastering the art of retrieving, filtering, and aggregating data using SQL commands and operators. Finally, you will document your journey, detailing the steps taken to ensure data accuracy and consistency, and showcasing your database's structure, population, and insights extracted through SQL queries. Through this comprehensive experience, you will develop a robust understanding of database management principles and practices.

Part I: Setting Up Your Database

Before you start structuring your database and adding data, you'll need to access your cPanel account, create a new database, configure user access, and access the newly created database.

- ☑ ~~Complete the Logging Into cPanel section to log into your cPanel account. Please use the credentials emailed to you by your instructor.~~
- ☑ ~~Complete the Creating A Database section to create a new database.~~
- ☑ ~~Complete the Creating A User section to set yourself up as a user of the database you created.~~
- ☑ ~~Complete the Accessing The Database section to access the database you created.~~

⚠️ Your **cPanel accounts do not support real-time collaboration** for MySQL databases, so **one group member** must **create, populate, and query the database under their own account**.

All group members should actively participate by guiding and assisting that member during setup, including creating tables, importing data, and running queries. Engaging in this process **ensures everyone understands the database structure, tables, and data integrity constraints**. Once complete, the database will be exported and uploaded to your group's shared repository so all members can access and import it into their own cPanel accounts.

Part II: Populating Your Database

It's crucial to ensure that the tables you create adhere to specific requirements to maintain data integrity and consistency. Follow these procedures to create tables, insert values manually, and import databases and tables:

- **Creating Tables With Integrity Constraints:** When creating tables, ensure that each table includes appropriate integrity constraints to enforce data accuracy and consistency. These constraints may include:
    - Primary key constraints to uniquely identify each record in the table and prevent duplicate entries.
    - Not null constraints to prohibit null values for essential attributes.
    - Unique constraints to ensure attribute values are distinct within the table.
    - Check constraints to enforce specific conditions on attribute values.
    - Foreign key constraints to maintain referential integrity between related tables.

- **Inserting Data With Validations:** When inserting data manually, validate that the values being inserted adhere to the defined integrity constraints. Ensure that:
    - Mandatory attributes are provided with non-null values.
    - Unique attribute values do not conflict with existing data.
    - Attribute values meet any specified conditions enforced by check constraints.
    - Primary key values are unique for each record in the table.
    - Foreign key constraints are satisfied by referencing existing values in related tables.

- **Importing Databases With Consistency:** If importing databases or tables, verify that the imported data conforms to the integrity constraints defined for the target database. Perform data validation checks to ensure:
    - Imported data meets the requirements specified by integrity constraints.
    - Integrity constraints are enforced consistently across the imported tables.
    - Primary key values are unique and consistent across the imported data.
    - Any foreign key relationships are maintained and referenced correctly.

Review the following to ensure that your database is populated accurately and consistently:

- ☑ ~~Review the Creating A Table section to design and create tables with appropriate integrity constraints.~~

- ☑ ~~Review the Inserting Data section to insert values into your tables while adhering to defined integrity constraints.~~

- ☑ ~~Review the Importing A Database section to import external data sources into your database while ensuring data consistency and integrity.~~

- ☑ ~~Review the Viewing A Database section to explore your populated database and verify data accuracy and completeness.~~

Part III: Querying Your Database

Now that your database is populated with relevant data, it's time to harness the power of SQL to retrieve and manipulate that data. You'll perform queries on single and multiple relations using various SQL commands and operators. Follow these steps to effectively query your database:

- **Single-Relation Queries**
  - Use the **SELECT** clause to retrieve specific columns or all columns from a single table.
  - Use the **FROM** clause to specify the table from which you want to retrieve data.
  - Apply the **WHERE** clause with comparison and logical operators to filter rows based on specific conditions.
  - Employ arithmetic operators (+, -, *, /) to perform calculations within your queries.
  - Use the **ORDER BY** clause to sort query results based on specified column(s).

- **Multiple-Relation Queries**
  - Perform joins (i.e., **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL OUTER JOIN**) to combine data from multiple tables based on related columns.
  - Use the **NATURAL JOIN** to join tables based on columns with the same name.
  - Apply aggregate functions (i.e., **SUM**, **AVG**, **COUNT**, **MIN**, **MAX**) to calculate summary statistics across groups of data.
  - Use the **GROUP BY** clause to group query results based on one or more columns.
  - Apply aggregate functions alongside **GROUP BY** to perform group-level calculations.
  - Use the **HAVING** clause to filter group-level results based on aggregate function results.

- **Executing Queries**
  - Write SQL queries that leverage the above-mentioned commands and operators to extract meaningful insights from your database.
  - Test your queries in a SQL environment.
  - Verify the accuracy and relevance of query results by cross-referencing them with the expected outcomes.

Review the following to perform queries on your database:

- ☐ Review the SQL Statements section to familiarize with common SQL statements to effectively communicate with your database.

- ☐ Review the Querying The Database section to write queries and ask questions about the data.

Part IV: Documentation

You will detail the steps taken to ensure data accuracy, demonstrate database population, and showcase meaningful insights extracted from the database.

1. Describe how you ensured that each table in your database includes appropriate integrity constraints to enforce data accuracy and consistency, and how you performed data validation checks when inserting data. Include details on:
    - Implementing primary key constraints to uniquely identify each record.
    - Applying not null constraints to essential attributes.
    - Enforcing unique constraints to ensure attribute distinctiveness.
    - Setting check constraints to enforce specific conditions on attribute values.
    - Maintaining referential integrity through foreign key constraints.

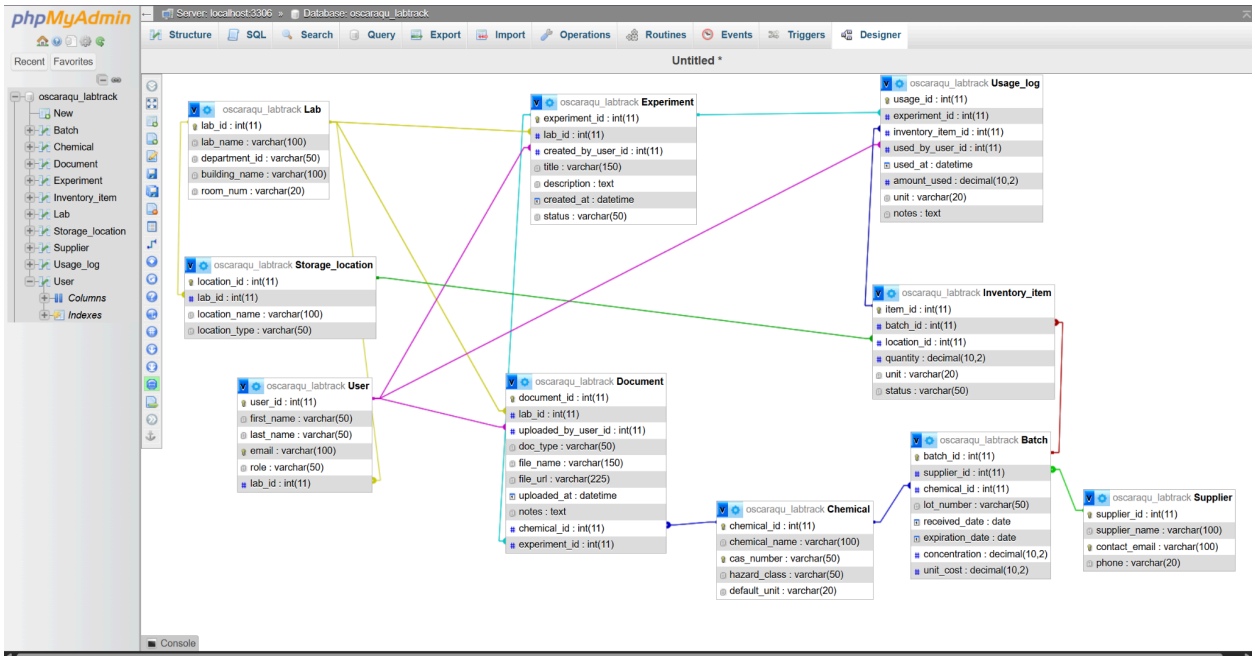
    Your description should outline the steps taken during the table creation process to ensure data integrity and accuracy, as well as the validation checks performed during data insertion.

To make sure that the data is accurate and consistent, we designed each table with integrity constraints as:

- **Primary key constraints:** For each table we assigned a primary key to uniquely identify each record and have no duplicates. These primary keys can be recognized by the *_id title.
- **Not null constraints:** We added not null to attributes that must definitely exist like names, emails, roles etc. This was done to prevent incomplete records from being created.
- **Unique constaints:** We used UNIQUE on attributes that should not repeat across records. Examples of these are email addresses for users, cas number for a chemical, and all the unique identifiers inside of the schema.
- **Check constraints:** We used CHECK to ensure reasonable values and business rules like:
    - Numeric values like amount and unit cost must be positive.
    - Restricted categories in classes that must have specific values for example hazard class needs to be valid. These checks stop invalid values from being in the database.

- **Foreign key constraints:** To maintain referential integrity, we created foreign keys so that the specific columns must refer to real rows in the parent table. Examples are Batch.chemical_id must firstly exist in the Chemical table and User.lab_id must exist in the Lab table.

2. Include screenshots of EACH table in your database. You can use the **Browse** tab and take a screenshot of the data that is displayed. This will demonstrate that you have successfully populated the database.



3. Five SQL queries were written to extract meaningful insights from the database. Each query serves a distinct purpose and uses different SQL clauses and operators including:

    - **Identify Flammable Chemicals:**

```
SELECT chemical_id, chemical_name, cas_number, hazard_class, default_unit
FROM Chemical
WHERE hazard_class = 'Flammable'
ORDER BY chemical_name;
```

Explanation: This query retrieves all chemicals classified as Flammable from the Chemical table. It helps lab personnel quickly identify hazardous materials that require special handling or storage precautions. The results are sorted alphabetically for easier review.

- **Calculate Cost of Purchasing 10 Batches of Ethanol**

```
SELECT c.chemical_name,
    s.supplier_name,
    MIN(b.unit_cost) AS cheapest_batch_cost,
    MIN(b.unit_cost) * 10 AS cost_10_batches
FROM Chemical c
JOIN Batch b ON b.chemical_id = c.chemical_id
JOIN Supplier s ON s.supplier_id = b.supplier_id
WHERE c.chemical_name = 'Ethanol'
GROUP BY c.chemical_name, s.supplier_name
ORDER BY cost_10_batches;
```

Explanation: This query determines the cheapest available batch of Ethanol per supplier and calculates the estimated cost of purchasing 10 batches. It uses joins to combine data from Chemical, Batch, and Supplier, applies the aggregate function MIN() to find the lowest unit cost, and performs arithmetic multiplication to compute total projected cost.

- **List Lab Managers and Their Lab Locations**

```
SELECT u.first_name, u.last_name, u.email, u.role,
    l.lab_name, l.building_name, l.room_num
FROM `User` u
JOIN Lab l ON u.lab_id = l.lab_id
WHERE u.role = 'Lab Manager';
```

Explanation: This query lists all users with the role of Lab Manager along with their associated lab name, building, and room number. It helps administrators identify leadership assignments and lab locations. The query uses a JOIN to connect User and Lab tables and filters results using a WHERE clause.

- **Count Number of Members in Each Lab**

```
SELECT l.lab_name, COUNT(u.lab_id) AS members
FROM Lab l
LEFT JOIN `User` u ON l.lab_id = u.lab_id
GROUP BY l.lab_id, l.lab_name
ORDER BY members DESC;
```

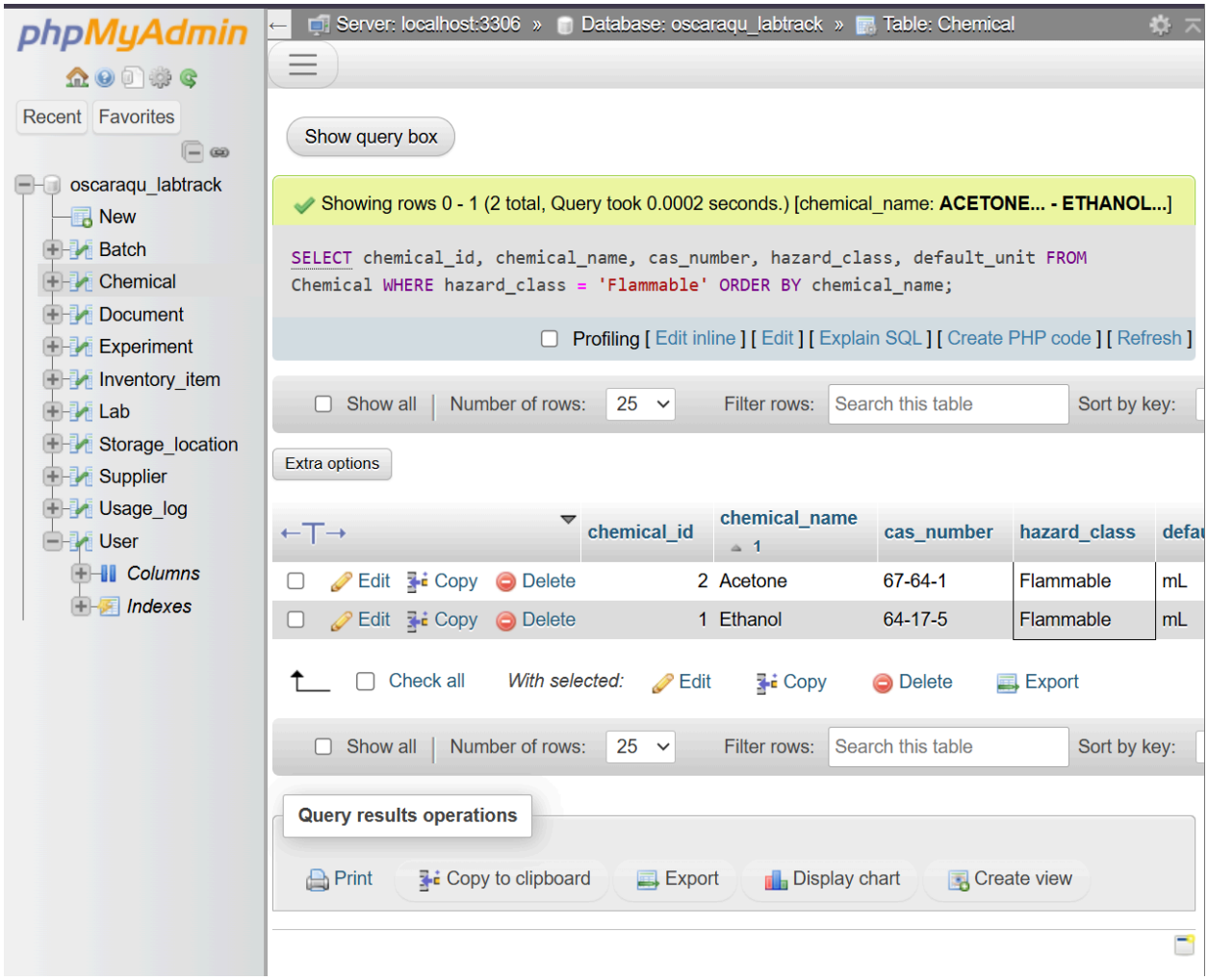
Explanation: This query calculates how many users are assigned to each lab. A LEFT JOIN ensures that labs with zero members are still included in the results. The COUNT() function aggregates user totals, and results are sorted from highest to lowest membership.

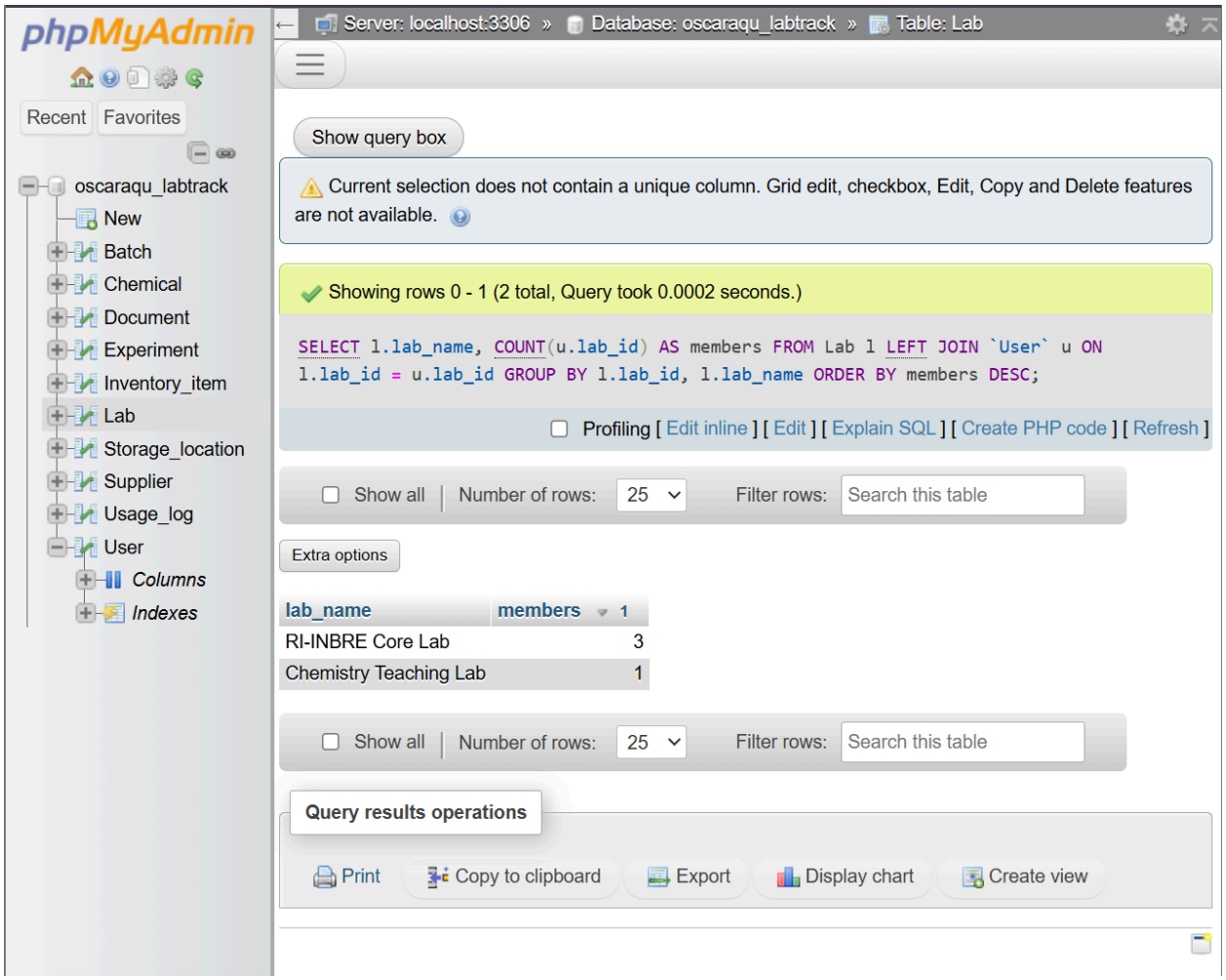
- **Summarize Chemical Usage by Experiment**

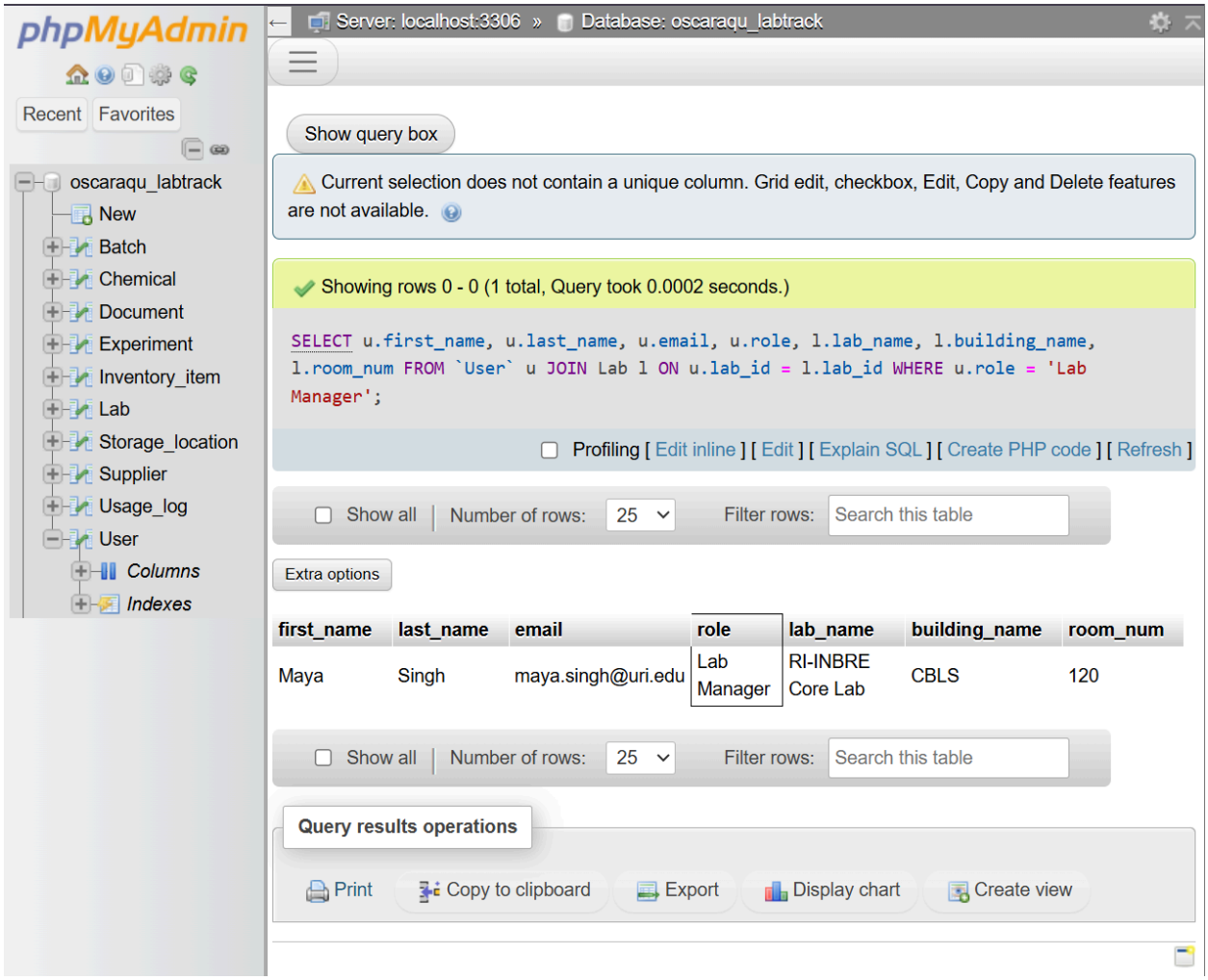
```
SELECT e.title AS experiment_title,
        ul.unit,
        SUM(ul.amount_used) AS total_used
FROM Usage_log ul
JOIN Experiment e ON ul.experiment_id = e.experiment_id
GROUP BY e.experiment_id, e.title, ul.unit
ORDER BY total_used DESC;
```

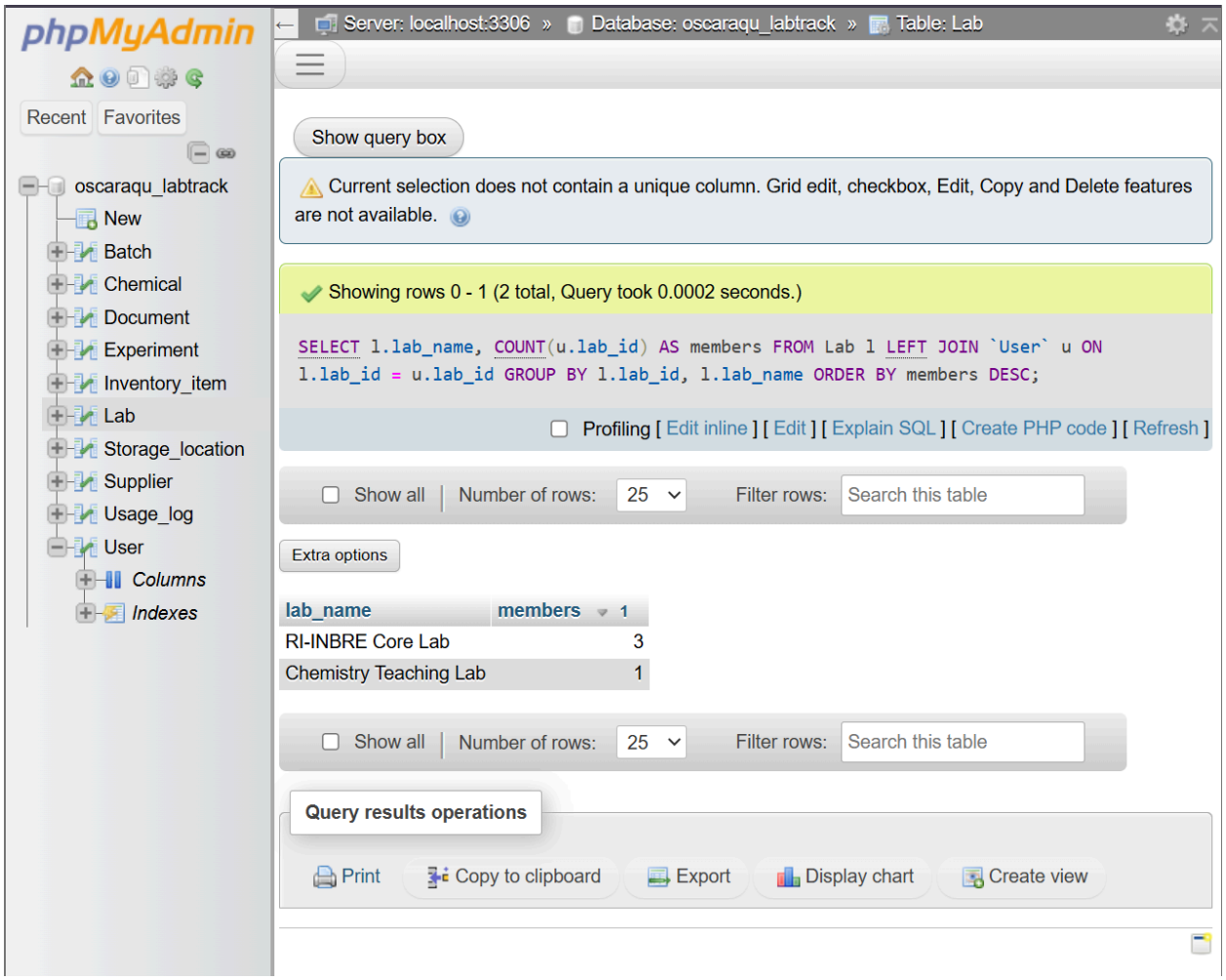
Explanation: This query summarizes total chemical usage per experiment. It aggregates the total amount used with SUM() and groups results by experiment and measurement unit to avoid mixing units. The output helps analyze which experiments consume the most resources

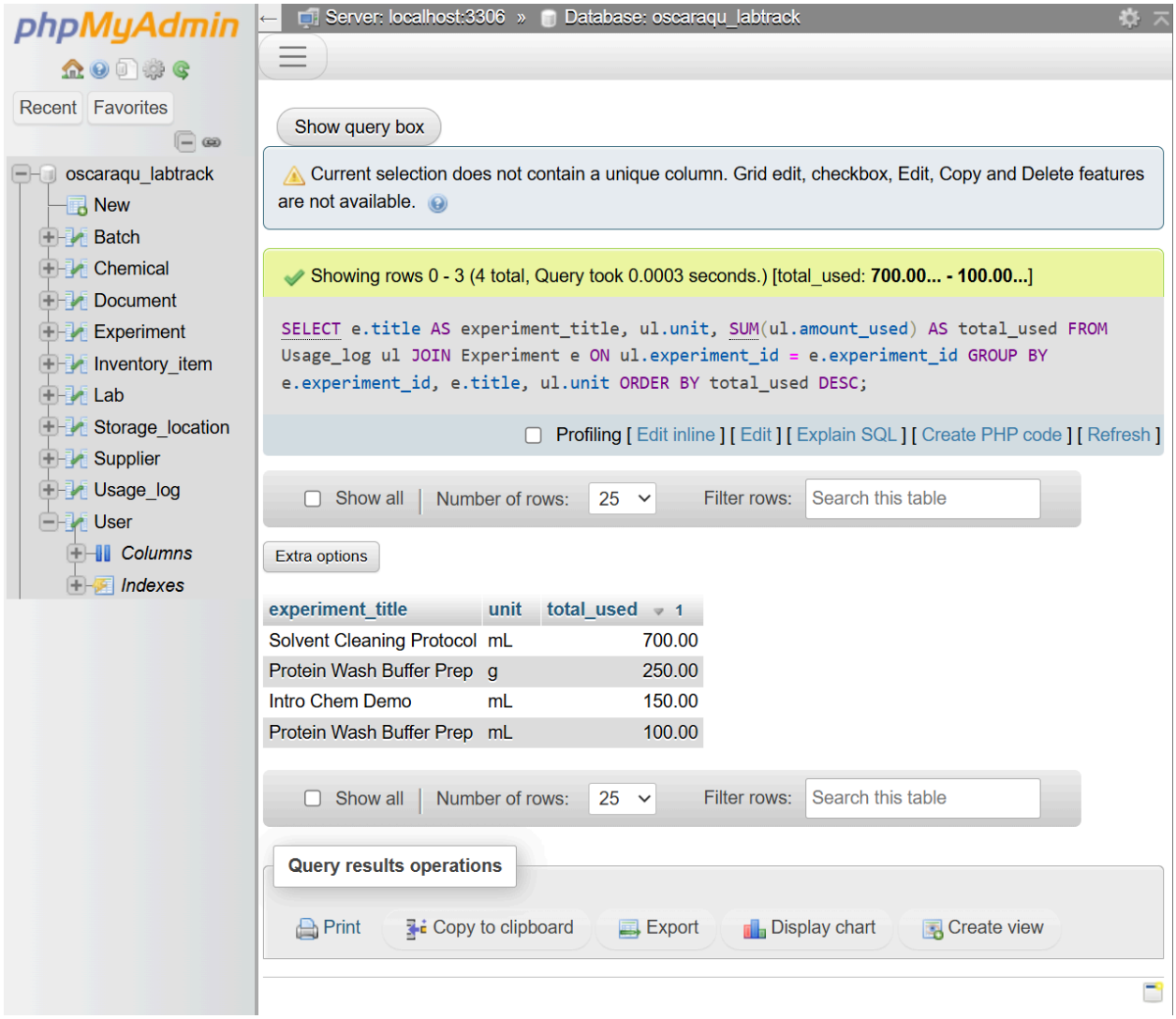
4. The results of each SQL query are included showcasing the insights extracted from the database. Screenshots of the tables displaying the query results are provided.


phpMyAdmin
Recent
Favorites
oscaraqu_labtrack
New
Batch
Chemical
Document
Experiment
Inventory_item
Lab
Storage_location
Supplier
Usage_log
User
Columns
Indexes
Server: localhost:3306 » Database: oscaraqu_labtrack » Table: Chemical
Show query box
Showing rows 0 - 1 (2 total, Query took 0.0002 seconds.) [chemical_name: ACETONE... - ETHANOL...]
SELECT chemical_id, chemical_name, cas_number, hazard_class, default_unit FROM Chemical WHERE hazard_class = 'Flammable' ORDER BY chemical_name;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table Sort by key:
Extra options
chemical_id
chemical_name
cas_number
hazard_class
Edit Copy Delete 2 Acetone 67-64-1 Flammable mL
Edit Copy Delete 1 Ethanol 64-17-5 Flammable mL
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table Sort by key:
Query results operations
Print Copy to clipboard Export Display chart Create view


phpMyAdmin
Recent
Favorites
oscaraqu_labtrack
New
Batch
Chemical
Document
Experiment
Inventory_item
Lab
Storage_location
Supplier
Usage_log
User
Columns
Indexes
Server: localhost:3306 » Database: oscaraqu_labtrack » Table: Lab
Show query box
Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.
Showing rows 0 - 1 (2 total, Query took 0.0002 seconds.)
SELECT l.lab_name, COUNT(u.lab_id) AS members FROM Lab l LEFT JOIN `User` u ON l.lab_id = u.lab_id GROUP BY l.lab_id, l.lab_name ORDER BY members DESC;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
lab_name members 1
RI-INBRE Core Lab 3
Chemistry Teaching Lab 1
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print
Copy to clipboard
Export
Display chart
Create view


phpMyAdmin
Recent
Favorites
oscaraqu_labtrack
New
Batch
Chemical
Document
Experiment
Inventory_item
Lab
Storage_location
Supplier
Usage_log
User
Columns
Indexes
Server: localhost:3306 » Database: oscaraqu_labtrack
Show query box
Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.
Showing rows 0 - 0 (1 total, Query took 0.0002 seconds.)
SELECT u.first_name, u.last_name, u.email, u.role, l.lab_name, l.building_name, l.room_num FROM `User` u JOIN Lab l ON u.lab_id = l.lab_id WHERE u.role = 'Lab Manager';
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
first_name last_name email role lab_name building_name room_num
Maya Singh maya.singh@uri.edu Lab Manager RI-INBRE Core Lab CBLS 120
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print
Copy to clipboard
Export
Display chart
Create view


phpMyAdmin
Recent
Favorites
oscaraqu_labtrack
New
Batch
Chemical
Document
Experiment
Inventory_item
Lab
Storage_location
Supplier
Usage_log
User
Columns
Indexes
Server: localhost:3306 » Database: oscaraqu_labtrack » Table: Lab
Show query box
Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.
Showing rows 0 - 1 (2 total, Query took 0.0002 seconds.)
SELECT l.lab_name, COUNT(u.lab_id) AS members FROM Lab l LEFT JOIN `User` u ON l.lab_id = u.lab_id GROUP BY l.lab_id, l.lab_name ORDER BY members DESC;
Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]
Show all | Number of rows: 25 Filter rows: Search this table
Extra options
lab_name members 1
RI-INBRE Core Lab 3
Chemistry Teaching Lab 1
Show all | Number of rows: 25 Filter rows: Search this table
Query results operations
Print
Copy to clipboard
Export
Display chart
Create view



Submission

When you're finished, complete the following steps to submit your work:

- ☑ ~~Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:~~
  - Google Docs (*File → Download → PDF Document*)
  - Microsoft Word (*File → Save As / Export → PDF*)
  - Pages (*File → Export To → PDF*)
- ☑ ~~Inside your repo, create a folder named **db** and export your database as an **SQL file** into that folder. The file should include all SQL code used to create tables, insert data, and~~

~~run other queries. Follow the steps in the Exporting A Database section to export your database into a single SQL file.~~

- [x] ~~All group members should then be able to pull this SQL file from the repo and upload it to their own cPanel accounts (see Importing A Database section).~~

- [x] ~~Upload all your changes to GitHub.~~

    - [ ] **If you're using GitHub Desktop (GUI)**, complete the Uploading Changes (GitHub Desktop) section to upload your changes from your local device to GitHub.

    - [ ] **If you're using Git (CLI)**, complete the Uploading Changes (GitHub CLI) section to upload your changes from your local device to GitHub.

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Populating & Querying Your Database grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.